

Campus Crisis Agent

Student Challenge Brief

STUDENT-FACING

Purpose

Build an Agentic AI system that processes campus reports in sequence and maintains an evolving understanding of incidents. A report is evidence. It is not necessarily a separate incident.

Your system should follow this behaviour: **Observe -> Correlate -> Assess -> Decide -> Act -> Record -> Monitor -> Reassess**. A chatbot connected to an LLM does not, by itself, meet the challenge.

What your agent must do

- Process each report in the supplied order.
- Decide whether it starts a new incident or updates an existing incident.
- Correlate duplicates and related evidence without relying on a shared incident ID.
- Maintain incident state, severity, confidence, services and action history.
- Reassess incidents when evidence conflicts or conditions change.
- Avoid unnecessary repeat dispatches.
- request human review when consequences or uncertainty justify it.
- recognise when an incident becomes controlled or resolved.

Required dashboard views

- 1 **Incoming Report:** the report currently being processed.
- 2 **Decision Log:** report ID, incident ID, relationship, severity, confidence, decision, service, status and concise reason.
- 3 **Incident Summary:** current type, location, severity, confidence, reports, services, status and action for every incident.
- 4 **Action History:** action, service, time and outcome/status for each incident.

Use any technology. Judges score clarity and observability, not visual polish.

Data supplied

- `campus_reports.csv`: 150 relatively clean development reports.
- `campus_services.csv`: fictional campus service directory.
- Output contract and example in the Technical and Submission Guide.

The unseen test contains 300 reports with duplicates, missing values, inconsistent labels, misspellings, conflicting accounts and malformed timestamps. The CSV remains technically parseable.

Deliverables by 13:00

- source code or repository snapshot;
- dependency list and exact run instructions;
- short architecture description;

- working dashboard;
- command or procedure that creates `predictions.jsonl`;
- completed team declaration naming each member's contribution.

Challenge-day schedule

- 09:00-09:15: briefing and file check.
- 09:15-12:40: build and test.
- 12:40-13:00: package, verify and submit.
- After 13:00: submission freeze, unseen testing, controlled replay and defence.

Rules

- Teams have three students.
- Do not access, request or attempt to infer the hidden test files.
- After freeze, do not change code, prompts, thresholds or dependencies.
- External AI services are allowed if your team supplies working access and stays within the announced event limits.
- The first valid frozen run is scored. A rerun is permitted only for an administrator-confirmed infrastructure failure.
- Explain all borrowed libraries, code and model use. Normal open-source components are allowed.

Frequently asked questions

Must we use an LLM? No. A rules engine, embeddings, classifiers, an LLM or a hybrid approach may be used.

Must our incident IDs match the hidden IDs? No. Correlation is scored as clustering behaviour.

Can the agent take no new action? Yes. Use an empty `actions` array when the existing response is already sufficient.

Can an incident become serious after minor reports? Yes. Several weak signals may collectively justify escalation.

Can a resolved incident reopen? Yes, if new evidence justifies it.

Are explanations automatically marked? No. Explanations support dashboard judging; automated scoring uses structured fields.